



# Informe Backend de Minimarket Plus

Asignatura: Desarrollo Backend II

| Campo       | Valor                                                                                                           |
|-------------|-----------------------------------------------------------------------------------------------------------------|
| Integrantes | Alonso Basualdo / Gustavo Domínguez                                                                             |
| Profesor    | Marcelo Zepeda                                                                                                  |
| Fecha       | 18/07/2026                                                                                                      |
| Repositorio | <a href="https://github.com/gustavoduocuc/minimarket-plus">https://github.com/gustavoduocuc/minimarket-plus</a> |

# Índice

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>1. Resumen técnico del avance .....</b>                    | <b>3</b>  |
| <b>2. Arquitectura de microservicios.....</b>                 | <b>5</b>  |
| 2.1 Módulos, puertos y responsabilidades.....                 | 5         |
| 2.2 Integración síncrona .....                                | 5         |
| <b>3. Frameworks de seguridad .....</b>                       | <b>6</b>  |
| 3.1 Auth (microservicio auth-service) .....                   | 7         |
| 3.2 Ventas (microservicio ventas-service).....                | 8         |
| 3.3 Catálogo (microservicio catalogo-inventario-service)..... | 9         |
| 3.4 Token interno servicio-a-servicio .....                   | 10        |
| <b>4. Pruebas unitarias e integración .....</b>               | <b>11</b> |
| 4.1 Stack .....                                               | 11        |
| 4.2 Cobertura de casos (conteo de clases *Test.java) .....    | 11        |
| 4.3 Ejecución y evidencia .....                               | 12        |
| <b>6. Documentación OAS y HATEOAS .....</b>                   | <b>15</b> |
| 6.1 OpenAPI (springdoc) .....                                 | 15        |
| 6.2 HATEOAS (HAL).....                                        | 18        |
| <b>7. Reflexión y conclusiones .....</b>                      | <b>20</b> |
| <b>8. Referencias.....</b>                                    | <b>20</b> |

# 1. Resumen técnico del avance

En las semanas previas (S1–S8) construimos MiniMarket Plus como API REST monolítica con Spring Boot 3: dominio de productos, inventario, carrito/ventas, notificaciones, seguridad JWT + RBAC, OpenAPI/HATEOAS y una batería creciente de tests.

Para la EFT dimos el salto a microservicios reales (tres procesos Spring Boot independientes, cada uno con su puerto y su base H2). El cliente ya no habla con un solo :8080:

autenticación vive en :8081, catálogo/inventario en :8082 y ventas/carrito en :8080.

Compartimos el mismo `jwt.secret` para que el token emitido por auth sea válido en los otros servicios.

Además desacoplamos el modelo de usuario en ventas: allí solo mantenemos una proyección (`id + username`), sin password ni roles locales. Los roles viajan en el JWT; la sincronización hacia ventas es best-effort vía endpoint interno y, si falla, un `ensure lazy` al primer uso del carrito/checkout. En checkout agregamos `tipoEntrega` (`RETIRO_EN_TIENDA` | `DESPACHO_DOMICILIO`) para reflejar el canal de entrega.

## Diagrama 1: Arquitectura anterior

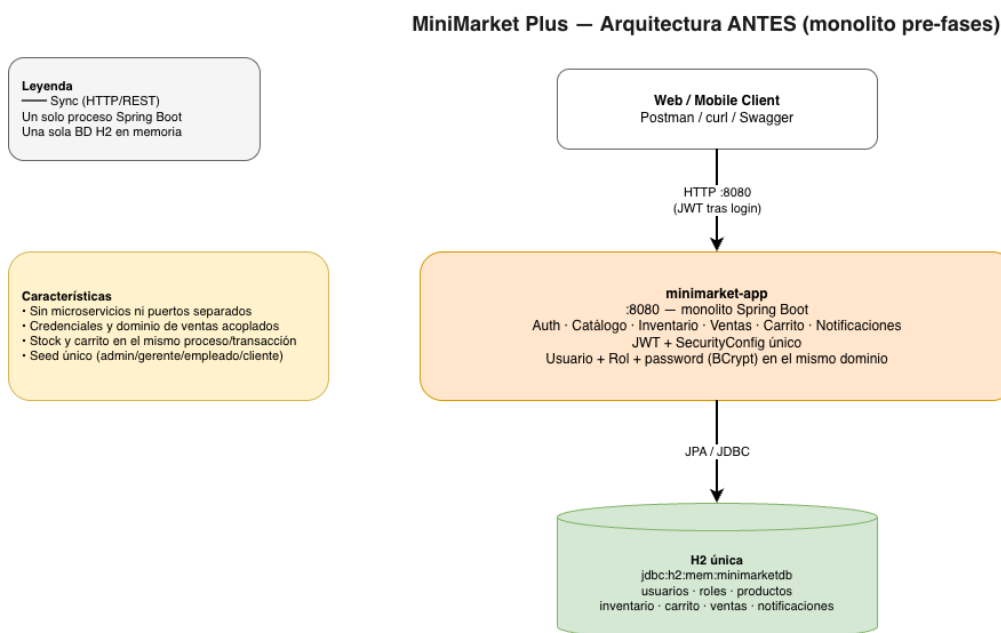
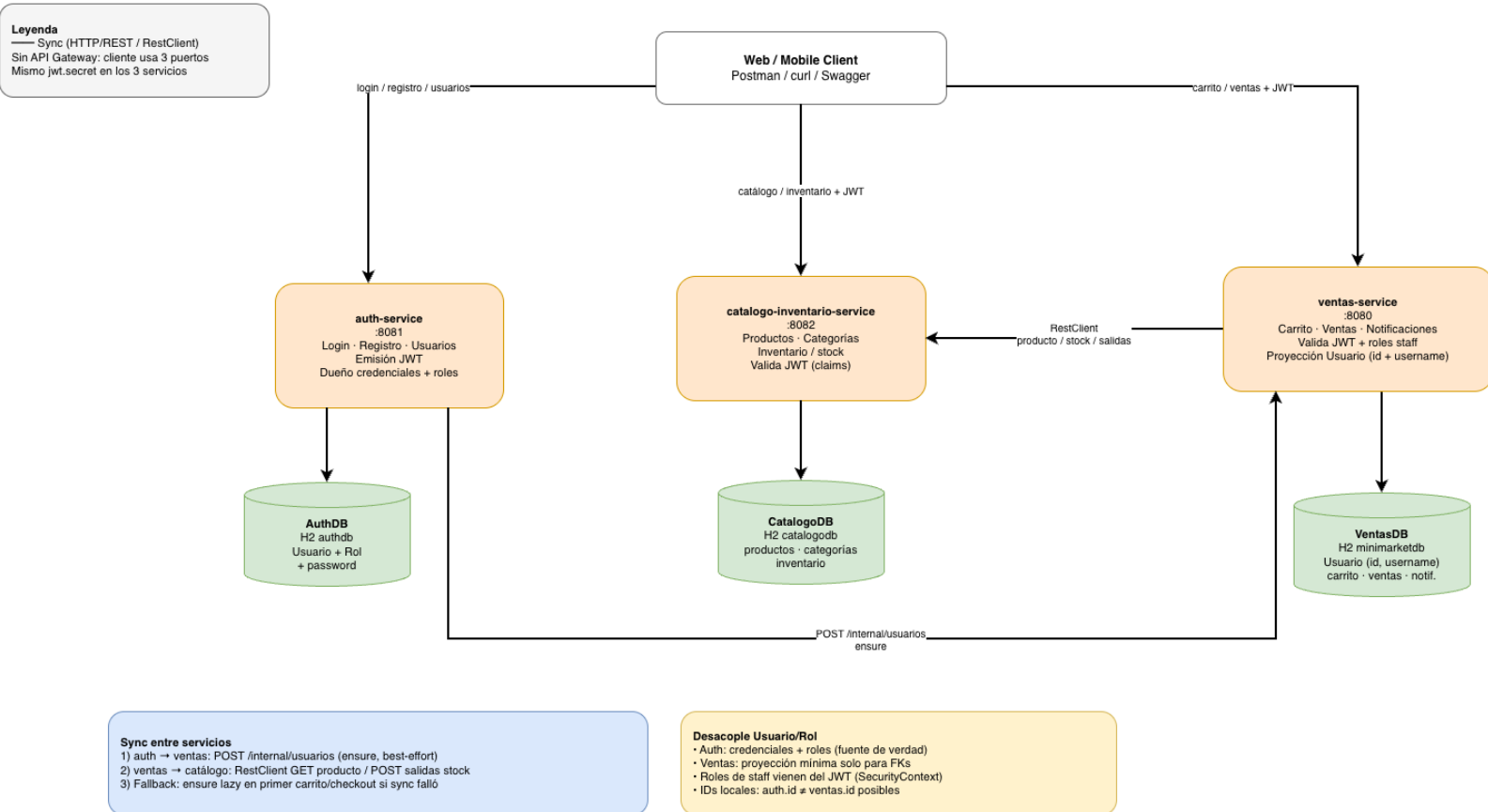


Diagrama 2: Arquitectura actual (tres microservicios + sync)

MiniMarket Plus — Arquitectura Actual (con microservicios reales)



## 2. Arquitectura de microservicios

### 2.1 Módulos, puertos y responsabilidades

| Módulo                      | Puerto | Responsabilidad                                                              |
|-----------------------------|--------|------------------------------------------------------------------------------|
| auth-service                | 8081   | Login, registro, usuarios, emisión JWT; sync best-effort hacia ventas        |
| catalogo-inventario-service | 8082   | Productos, categorías, inventario (consultas y movimientos de stock)         |
| ventas-service              | 8080   | Carrito, ventas, notificaciones; valida JWT; consume catálogo vía RestClient |

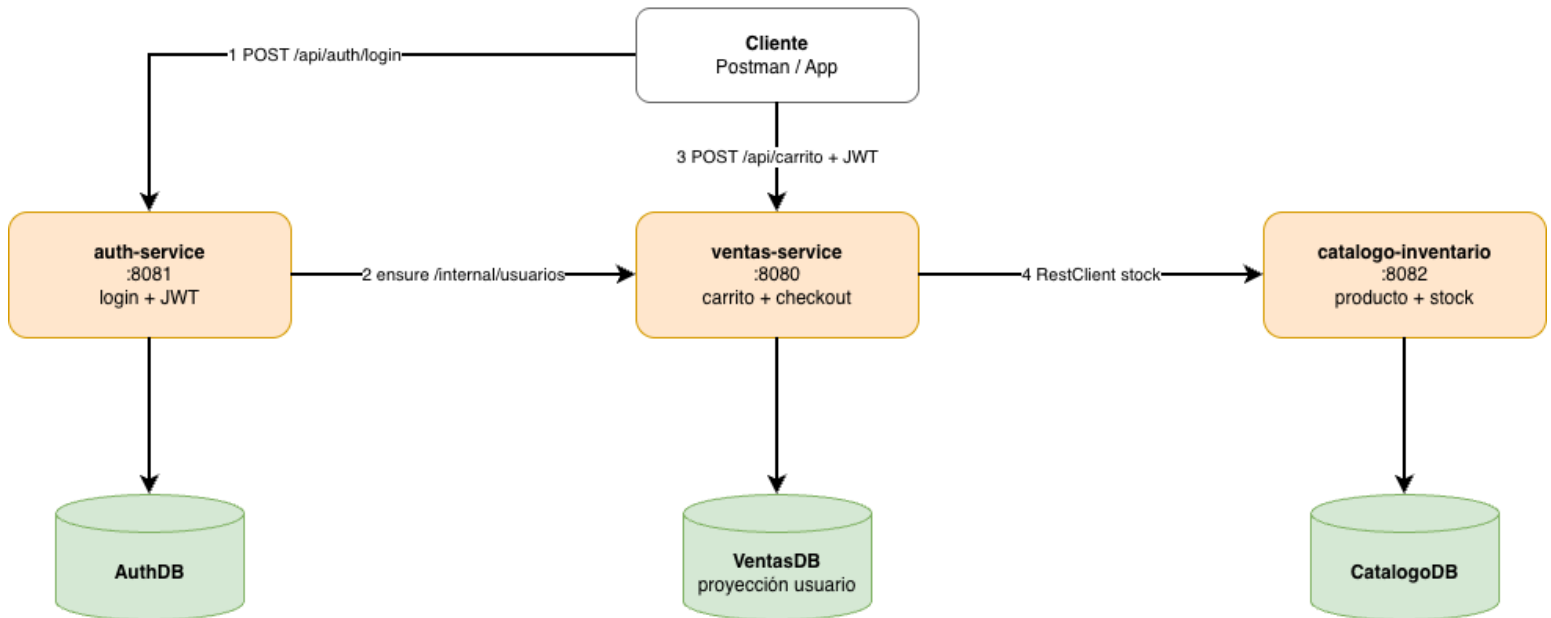
El cliente (App/Postman) apunta a cada puerto. La documentación de arranque y el flujo curl E2E están en el [README.md](#) del repositorio.

### 2.2 Integración síncrona

1. Login → POST `http://localhost:8081/api/auth/login` → respuesta con JWT.
2. Registro / alta de usuario → auth llama a ventas POST `/internal/usuarios` con header `X-Internal-Token` (best-effort).
3. Carrito / checkout → Bearer JWT contra ventas; si no existe proyección local, `UsuarioService.ensure(username)`.
4. Stock en checkout → ventas → catálogo con RestClient (`catalogo.base-url`, por defecto `http://localhost:8082`) para validar producto y registrar salida de inventario.

Diagrama 3:

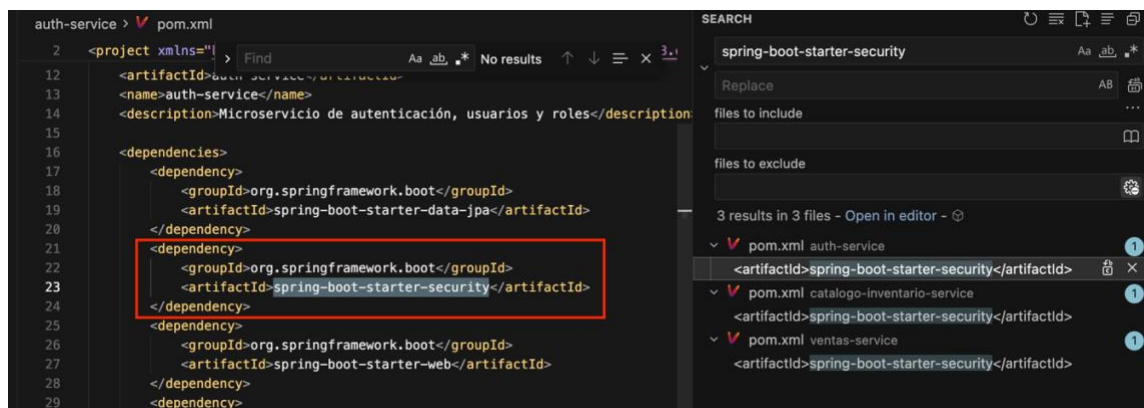
## MiniMarket Plus — Flujo E2E: login, ensure, carrito y checkout



### 3. Frameworks de seguridad

Luego de agregar la dependencia `spring-boot-starter-security` en `pom.xml`, configuramos Spring Security en modo stateless con JWT y autorización por roles (RBAC). Cada servicio declara su propio `SecurityConfig` (cadena de filtros, CSP, frame options, matchers).

**Figura 1: Dependencia `spring-boot-starter-security` en `pom.xml` de cada microservicio**



### 3.1 Auth (microservicio auth-service)

- Rutas públicas: /api/auth/\*\*, Swagger, /public/\*\*.
- Gestión de usuarios: /api/usuarios/\*\* → rol ADMIN.
- Filtros: rate limit de login + JwtRequestFilter.
- Headers: frameOptions.deny(), CSP default-src 'self'.
- Política de contraseñas y logout de login documentados en [docs/seguridad-acceso-y-datos.md](#).

Figura 2: SecurityConfig de auth (RBAC + filtros JWT/rate-limit)

```
auth-service > src > main > java > com > minimarket > security > config > SecurityConfig.java > ...
2 -/dev/duoc/backend-2/minimarket-plus/auth-service - Contains emphasized items
41
42 @Bean
43 public PasswordEncoder passwordEncoder() {
44     return new org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder();
45 }
46
47 @Bean
48 @Order(1)
49 @Profile("dev")
50 public SecurityFilterChain h2ConsoleSecurityFilterChain(HttpSecurity http) throws Exception {
51     http
52         .securityMatcher(PathRequest.toH2Console())
53         .csrf(csrf -> csrf.disable())
54         .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
55         .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
56     return http.build();
57 }
58
59 @Bean
60 @Order(2)
61 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
62     http
63         .csrf(csrf -> csrf.disable())
64         .formLogin(form -> form.disable())
65         .httpBasic(basic -> basic.disable())
66         .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
67         .exceptionHandling(exception -> exception
68             .authenticationEntryPoint(securityAuditHandler)
69             .accessDeniedHandler(securityAuditHandler)
70         )
71         .headers(headers -> headers
72             .frameOptions(frame -> frame.deny())
73             .contentSecurityPolicy(csp -> csp.policyDirectives(policyDirectives: "default-src 'self'"))
74         )
75         .authorizeHttpRequests(auth -> auth
76             .requestMatchers(...patterns: "/api/auth/**").permitAll()
77             .requestMatchers(...patterns: "/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll()
78             .requestMatchers(...patterns: "/api/usuarios/**").hasRole(role: "ADMIN")
79             .anyRequest().authenticated()
80         );
81
82     http.addFilterBefore(loginRateLimitFilter, beforeFilter: UsernamePasswordAuthenticationFilter.class);
83     http.addFilterBefore(jwtRequestFilter, beforeFilter: UsernamePasswordAuthenticationFilter.class);
84
85     return http.build();
86 }
87
88 }
```

### 3.2 Ventas (microservicio ventas-service)

- Carrito propio: roles CLIENTE / ADMIN (y staff en altas según matchers).
- Checkout ajeno / listados de carrito / ventas: roles de staff (EMPLEADO, GERENTE, ADMIN).
- Mismo JWT secret compartido con auth y catálogo.

Figura 3: SecurityConfig de ventas (RBAC carrito/ventas)

```

ventas-service > src > main > java > com > minimarket > security > config > SecurityConfig.java > ...
19 public class SecurityConfig {
20     }
21
22     @Bean
23     @Order(1)
24     @Profile("dev")
25     public SecurityFilterChain h2ConsoleSecurityFilterChain(HttpSecurity http) throws Exception {
26         http
27             .securityMatcher(PathRequest.toH2Console())
28             .csrf(csrf -> csrf.disable())
29             .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
30             .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
31         return http.build();
32     }
33
34     @Bean
35     @Order(2)
36     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
37         http
38             .csrf(csrf -> csrf.disable())
39             .formLogin(form -> form.disable())
40             .httpBasic(basic -> basic.disable())
41             .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
42             .exceptionHandling(exception -> exception
43                 .authenticationEntryPoint(securityAuditHandler)
44                 .accessDeniedHandler(securityAuditHandler)
45             )
46             .headers(headers -> headers
47                 .frameOptions(frame -> frame.deny())
48                 .contentSecurityPolicy(csp -> csp.policyDirectives(policyDirectives: "default-src 'self'"))
49             )
50             .authorizeHttpRequests(auth -> auth
51                 .requestMatchers(...patterns: "/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll()
52                 .requestMatchers(...patterns: "/internal/**").permitAll()
53                 .requestMatchers(HttpMethod.POST, ...patterns: "/api/carrito/checkout/*")
54                     .hasAnyRole(...roles: "EMPLEADO", "GERENTE", "ADMIN")
55                 .requestMatchers(HttpMethod.POST, ...patterns: "/api/carrito/checkout").authenticated()
56                 .requestMatchers(HttpMethod.POST, ...patterns: "/api/carrito")
57                     .hasAnyRole(...roles: "CLIENTE", "EMPLEADO", "GERENTE", "ADMIN")
58                 .requestMatchers(HttpMethod.GET, ...patterns: "/api/carrito/todos", "/api/carrito/*")
59                     .hasAnyRole(...roles: "EMPLEADO", "GERENTE", "ADMIN")
60                 .requestMatchers(...patterns: "/api/carrito/**").hasAnyRole(...roles: "CLIENTE", "ADMIN")
61                 .requestMatchers(...patterns: "/api/ventas/**", "/api/detalle-ventas/**").hasAnyRole(...roles: "EMPLEADO", "GERENTE", "ADMIN")
62                 .requestMatchers(HttpMethod.POST, ...patterns: "/api/notificaciones/promociones").hasAnyRole(...roles: "GERENTE", "ADMIN")
63                 .requestMatchers(...patterns: "/api/notificaciones/**").authenticated()
64                 .anyRequest().authenticated()
65             );
66
67         http.addFilterBefore(jwtRequestFilter, beforeFilter: UsernamePasswordAuthenticationFilter.class);
68
69         return http.build();
70     }
71 }

```

### 3.3 Catálogo (microservicio catalogo-inventario-service)

- GET públicos de productos/categorías para permitir a usuarios en línea listar los productos.
- Inventario y mutaciones de catálogo restringidos a roles de staff/gerente/admin según matcher HTTP.

Figura 4: SecurityConfig de Inventario

```

catalogo-inventario-service > src > main > java > com > minimarket > security > config > SecurityConfig.java > ...
19 public class SecurityConfig {
34     public SecurityFilterChain h2ConsoleSecurityFilterChain(HttpSecurity http) throws
35     {
36         .securityMatcher(PathRequest.toH2Console())
37         .csrf(csrf -> csrf.disable())
38         .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
39         .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
40         return http.build();
41     }
42
43     @Bean
44     @Order(2)
45     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
46         http
47             .csrf(csrf -> csrf.disable())
48             .formLogin(form -> form.disable())
49             .httpBasic(basic -> basic.disable())
50             .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
51             .exceptionHandling(exception -> exception
52                 .authenticationEntryPoint(securityAuditHandler)
53                 .accessDeniedHandler(securityAuditHandler)
54             )
55             .headers(headers -> headers
56                 .frameOptions(frame -> frame.deny())
57                 .contentSecurityPolicy(csp -> csp.policyDirectives(policyDirectives: "default-src 'self'"))
58             )
59             .authorizeHttpRequests(auth -> auth
60                 .requestMatchers(...patterns: "/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll()
61                 .requestMatchers(HttpMethod.GET, ...patterns: "/api/productos/**", "/api/categorias/**").permitAll()
62                 .requestMatchers(HttpMethod.GET, ...patterns: "/api/inventario/**").hasAnyRole(...roles: "EMPLEADO", "GERENTE", "ADMIN")
63                 .requestMatchers(HttpMethod.POST, ...patterns: "/api/inventario/salidas").hasAnyRole(...roles: "EMPLEADO", "GERENTE", "ADMIN")
64                 .requestMatchers(HttpMethod.POST, ...patterns: "/api/inventario/entradas").hasAnyRole(...roles: "GERENTE", "ADMIN")
65                 .requestMatchers(HttpMethod.POST, ...patterns: "/api/inventario/**").hasAnyRole(...roles: "GERENTE", "ADMIN")
66                 .requestMatchers(HttpMethod.PUT, ...patterns: "/api/inventario/**").hasAnyRole(...roles: "GERENTE", "ADMIN")
67                 .requestMatchers(HttpMethod.DELETE, ...patterns: "/api/inventario/**").hasAnyRole(...roles: "GERENTE", "ADMIN")
68                 .requestMatchers(HttpMethod.POST, ...patterns: "/api/productos/**", "/api/categorias/**").hasAnyRole(...roles: "GERENTE",
69                     "ADMIN")
70                 .requestMatchers(HttpMethod.PUT, ...patterns: "/api/productos/**", "/api/categorias/**").hasAnyRole(...roles: "GERENTE",
71                     "ADMIN")
72                 .requestMatchers(HttpMethod.DELETE, ...patterns: "/api/productos/**", "/api/categorias/**").hasAnyRole(...roles: "GERENTE",
73                     "ADMIN")
74                 .anyRequest().authenticated()
75             );
76         http.addFilterBefore(jwtRequestFilter, beforeFilter: UsernamePasswordAuthenticationFilter.class);
77         return http.build();
78     }

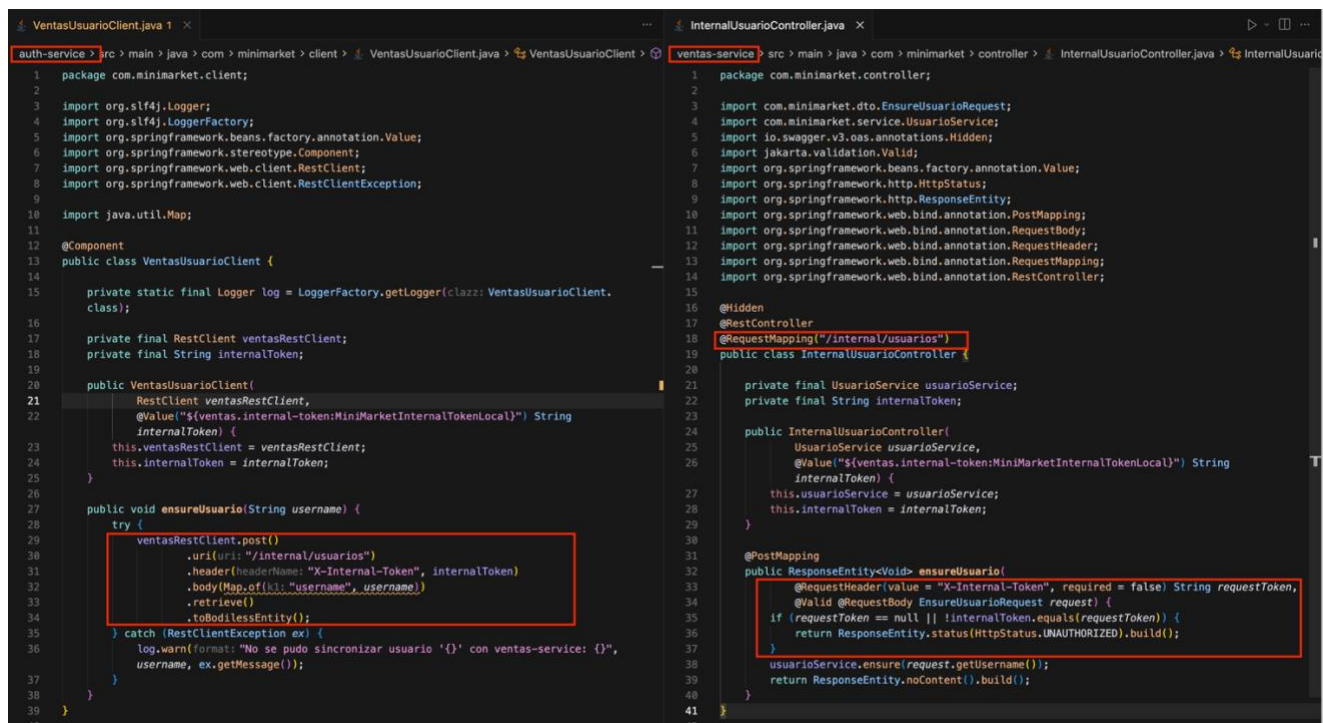
```

### 3.4 Token interno servicio-a-servicio

Implementamos sincronización entre microservicios: cuando un usuario se registra en auth-service, este notifica a ventas-service para persistir una proyección local del usuario y garantizar integridad referencial en carritos y ventas.

Auth → ventas usa `VentasUsuarioClient` con header `X-Internal-Token`. Ventas valida en `InternalUsuarioController` antes de `usuarioService.ensure(...)`.

Figura 5: Endpoint interno de proyección de usuario



## 4. Pruebas unitarias e integración

### 4.1 Stack

| Herramienta | Uso                                                       | Dependencia(s) en pom.xml                                        |
|-------------|-----------------------------------------------------------|------------------------------------------------------------------|
| JUnit 5     | Tests unitarios e integración                             | spring-boot-starter-test                                         |
| Mockito     | Dobles de servicios/clientes                              | spring-boot-starter-test                                         |
| MockMvc     | Controllers y seguridad HTTP                              | spring-boot-starter-test                                         |
| WireMock    | Simular catálogo desde ventas (RestClient)                | wiremock-standalone (org.wiremock)                               |
| JaCoCo      | Cobertura por módulo + reporte agregado (coverage-report) | No es dependency de código. Es plugin Maven: jacoco-maven-plugin |

### 4.2 Cobertura de casos (conteo de clases \*Test.java)

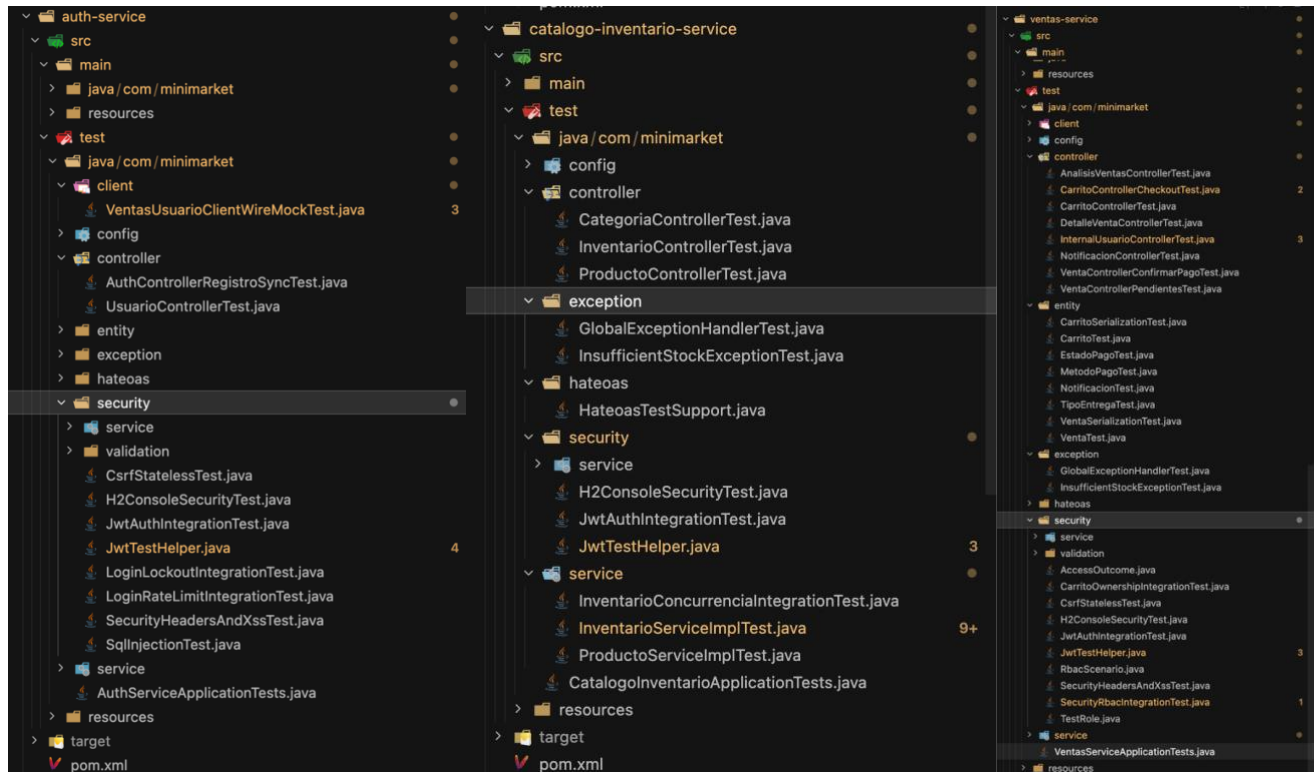
| Módulo                      | Clases de test (aprox.) |
|-----------------------------|-------------------------|
| auth-service                | 16                      |
| catalogo-inventario-service | 11                      |
| ventas-service              | 37                      |
| Total                       | 64                      |

#### Clases más representativas dentro del flow E2E:

- Seguridad / RBAC: `JwtAuthIntegrationTest`, `LoginLockoutIntegrationTest`, `CsrfStatelessTest`, `H2ConsoleSecurityTest`, `SqlInjectionTest`.
- Sync interno: `InternalUsuarioControllerTest`, `AuthControllerRegistroSyncTest`.

- Checkout / entrega: CarritoCheckoutServiceImplTest, TipoEntregaTest.
- Integración catálogo: tests con WireMock en ventas (RestClient stock).
- OpenAPI: OpenApiIntegrationTest (ventas).

**Figura 6: Estructura testing de cada microservicio**



## 4.3 Ejecución y evidencia

Desde la raíz del repo:

```
./mvnw clean test
./mvnw clean verify
```

El `verify` genera reportes JaCoCo por módulo y el agregado en:

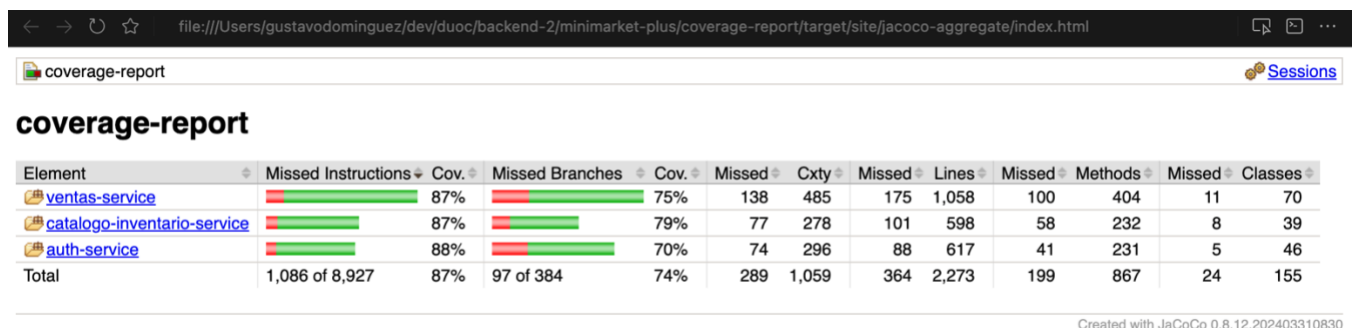
`coverage-report/target/site/jacoco-aggregate/index.html`

(detalle en [docs/coverage.md](#)).

Figura 7: Salida de ./mvnw clean test (BUILD SUCCESS)

```
2026-07-18T16:41:58.612-04:00 WARN 60150 --- [main] L.W.Exception.GlobalExceptionHandler : Solicitud inválida: detalle interno
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.236 s -- in com.minimarket.exception.GlobalExceptionHandlerTest
[INFO] Running com.minimarket.client.HttpProductoServiceWireMockTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s -- in com.minimarket.client.HttpProductoServiceWireMockTest
[INFO] Running com.minimarket.client.HttpInventarioServiceWireMockTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.028 s -- in com.minimarket.client.HttpInventarioServiceWireMockTest
[INFO] Results:
[INFO] Tests run: 199, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----[ pom ]-----
[INFO] Building coverage-report 0.0.1-SNAPSHOT [5/5]
[INFO] from coverage-report/pom.xml
[INFO]
[INFO] --- jacoco:0.8.12:prepare-agent (prepare-agent) @ coverage-report ---
[INFO] argLine set to -javaagent:/Users/gustavodominguez/.m2/repository/org/jacoco/org.jacoco.agent/0.8.12/org.jacoco.agent-0.8.12-runtime.jar=destfile=/Users/gustavodominguez/dev/duo
c/backend-2/minimarket-plus/coverage-report/target/jacoco.exec
[INFO]
[INFO] Reactor Summary for minimarket-plus 0.0.1-SNAPSHOT:
[INFO]
[INFO] minimarket-plus ..... SUCCESS [ 0.001 s]
[INFO] auth-service ..... SUCCESS [ 15.203 s]
[INFO] catalogo-inventario-service ..... SUCCESS [ 9.277 s]
[INFO] ventas-service ..... SUCCESS [ 13.170 s]
[INFO] coverage-report ..... SUCCESS [ 0.039 s]
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 37.881 s
[INFO] Finished at: 2026-07-18T16:41:59-04:00
[INFO]
```

Figura 8: Reporte JaCoCo agregado



## 5. Mejoras aplicadas (respecto al monolito)

| Mejora                          | Qué resolvimos                                                       |
|---------------------------------|----------------------------------------------------------------------|
| Split a 3 microservicios        | Separación de despliegue y bases; límites claros de responsabilidad  |
| JWT compartido                  | Auth emite; ventas y catálogo validan sin gateway                    |
| Proyección de usuario en ventas | Sin password/roles locales; IDs locales; sync + ensure lazy          |
| Staff vía JWT                   | Autorización por authorities del token, no por entidad Rol en ventas |

|                        |                                                                                     |
|------------------------|-------------------------------------------------------------------------------------|
| Checkout tipoEntrega   | RETIRO_EN_TIENDA (default) o DESPACHO_DOMICILIO en request/response y entidad Venta |
| Documentación en /docs | Seguridad, estados de venta, cobertura, colección Postman, diagramas Draw.io        |

**Figura 9: Checkout con tipoEntrega (código + Postman)**

```

ventas-service > src > main > java > com > minimarket > dto > CheckoutRequest.java > {} com.minimarket.dto
1  package com.minimarket.dto;
2
3  import com.minimarket.entity.MetodoPago;
4  import com.minimarket.entity.TipoEntrega;
5  import io.swagger.v3.oas.annotations.media.Schema;
6
7  public class CheckoutRequest {
8
9      @Schema(description = "Método de pago de la venta", example = "EFECTIVO")
10     private MetodoPago metodoPago;
11
12     @Schema(description = "Tipo de entrega (opcional; default RETIRO_EN_TIENDA)", example = "DESPACHO_DOMICILIO")
13     private TipoEntrega tipoEntrega;
14
15     public MetodoPago getMetodoPago() {
16         return metodoPago;
17     }
18
19     public void setMetodoPago(MetodoPago metodoPago) {
20         this.metodoPago = metodoPago;
21     }
22
23     public TipoEntrega getTipoEntrega() {
24         return tipoEntrega;
25     }
26
27     public void setTipoEntrega(TipoEntrega tipoEntrega) {
28         this.tipoEntrega = tipoEntrega;
29     }
30 }
31

```

MiniMarket Plus API Copy > Carrito > Checkout del carrito

POST `{{base_url}}/api/carrito/checkout` Send

Docs Params Authorization Headers 10 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```

1  {
2    "metodoPago": "EFECTIVO",
3    "tipoEntrega": "DESPACHO_DOMICILIO"
4  }

```

Body 201 Created 99 ms 501 B Save Response

**JSON** Preview Visualize

```

1  {
2    "ventaId": 1,
3    "metodoPago": "EFECTIVO",
4    "estadoPago": "PENDIENTE_PAGO",
5    "tipoEntrega": "DESPACHO_DOMICILIO",
6    "total": 2400.0
7  }

```

**Code snippet**

cURL

```

1  curl --location 'http://localhost:8080/api/carrito/checkout' \
2    --header 'Content-Type: application/json' \
3    --header 'Authorization: ..... ' \
4    --data '{
5      "metodoPago": "EFECTIVO",
6      "tipoEntrega": "DESPACHO_DOMICILIO"
7    }'

```

## 6. Documentación OAS y HATEOAS

### 6.1 OpenAPI (springdoc)

Cada servicio expone su propio Swagger UI:

| Servicio | URL                                   |
|----------|---------------------------------------|
| Auth     | http://localhost:8081/swagger-ui.html |
| Catálogo | http://localhost:8082/swagger-ui.html |
| Ventas   | http://localhost:8080/swagger-ui.html |

Configuración vía OpenApiConfig en cada módulo (auth-service, catalogo-inventario-service, ventas-service).

**Figura 10: Swagger UI — auth (:8081)**

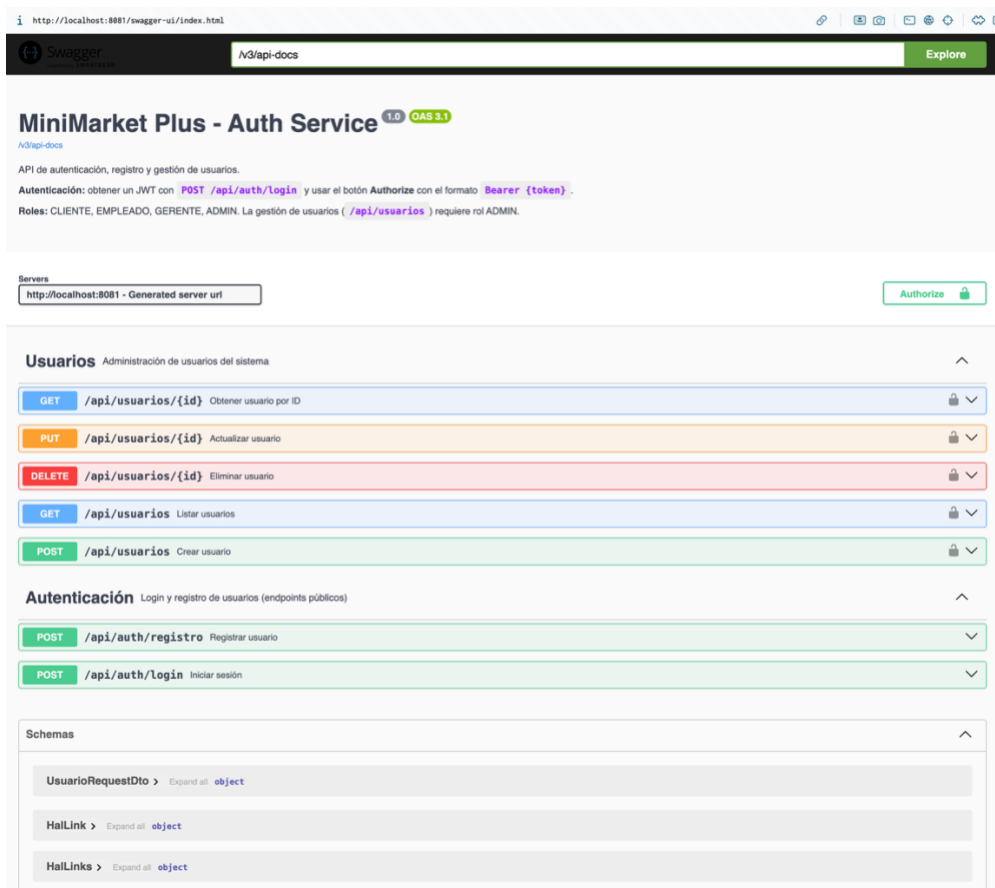


Figura 11: Swagger UI — catálogo (:8082)

Swagger UI interface for the MiniMarket Plus API. The browser address bar shows `http://localhost:8082/swagger-ui/index.html#/Productos`. The Swagger logo and version `1.0 OAS 3.1` are visible. The API title is **MiniMarket Plus - Catálogo e Inventario**. The description states: "API de productos, categorías e inventario. Autenticación: JWT emitido por `auth-service` ( `POST /api/auth/login` en puerto 8081). Usar `Authorize` con `Bearer {token}`. GET de productos/categorías es público. Mutaciones e inventario requieren roles de staff."

**Servers**  
`http://localhost:8082 - Generated server url` [Authorize](#)

### Categorías

Categorías del catálogo

- GET** `/api/categorias/{id}` Obtener categoría por ID
- PUT** `/api/categorias/{id}` Actualizar categoría
- DELETE** `/api/categorias/{id}` Eliminar categoría
- GET** `/api/categorias` Listar categorías
- POST** `/api/categorias` Crear categoría

### Productos

Catálogo de productos

- GET** `/api/productos/{id}` Obtener producto por ID
- PUT** `/api/productos/{id}` Actualizar producto
- DELETE** `/api/productos/{id}` Eliminar producto
- GET** `/api/productos` Listar productos
- POST** `/api/productos` Crear producto
- GET** `/api/productos/{id}/stock` Consultar stock disponible de un producto

### Inventario

Movimientos de inventario

- GET** `/api/inventario/{id}` Obtener movimiento por ID
- PUT** `/api/inventario/{id}` Actualizar movimiento de inventario
- DELETE** `/api/inventario/{id}` Eliminar movimiento de inventario
- GET** `/api/inventario` Listar movimientos de inventario
- POST** `/api/inventario` Registrar movimiento de inventario
- POST** `/api/inventario/salidas` Registrar salida de stock
- POST** `/api/inventario/entradas` Registrar entrada de stock

Figura 12: Swagger UI — ventas (:8080)

Swagger UI interface for the **MiniMarket Plus API — ventas-service** (version 1.0 OAS 3.1).

API REST de ventas, carrito y notificaciones ( **ventas-service** :8080).

Catálogo e inventario: **catalogo-inventario-service** (puerto 8082). Autenticación: JWT desde **auth-service** ( **POST /api/auth/login** en puerto 8081).

Roles:

- **CLIENTE**: carrito propio y checkout autogestionado.
- **EMPLEADO**: operaciones de venta y carrito de terceros.
- **GERENTE**: operaciones de staff y promociones.
- **ADMIN**: acceso total a operaciones de negocio en este servicio.

**HATEOAS (HAL)**: las respuestas de Carrito y Venta usan Hypertext Application Language.

- Recurso individual ( **EntityModel** ): campos del dominio + **\_links**.
- Colección ( **CollectionModel** ): **\_embedded.<nombreLista>** + **\_links** de la colección.
- Los enlaces de mutación solo aparecen según el rol del JWT.

**Servers**

http://localhost:8080 - Generated server url

**Authorize**

### Detalle de ventas

Lineas de detalle de ventas

- GET** **/api/detalle-ventas/{id}** Obtener detalle de venta por ID
- PUT** **/api/detalle-ventas/{id}** Actualizar detalle de venta
- DELETE** **/api/detalle-ventas/{id}** Eliminar detalle de venta
- GET** **/api/detalle-ventas** Listar detalles de venta
- POST** **/api/detalle-ventas** Crear detalle de venta

### Análisis de Ventas

Reportes y rankings de ventas

- GET** **/api/ventas/analisis/productos-mas-vendidos** Productos más vendidos

### Carrito

Gestión del carrito de compras y checkout

- GET** **/api/carrito** Obtener carrito propio
- POST** **/api/carrito** Agregar producto al carrito
- DELETE** **/api/carrito** Vaciar carrito
- POST** **/api/carrito/checkout** Checkout del carrito propio
- POST** **/api/carrito/checkout/{usuarioId}** Checkout de carrito de un tercero
- GET** **/api/carrito/{id}** Obtener carrito por ID
- GET** **/api/carrito/todos** Listar todos los carritos
- DELETE** **/api/carrito/items/{productoId}** Quitar producto del carrito

### Ventas

Gestión de ventas y confirmación de pagos

- GET** **/api/ventas** Listar ventas
- POST** **/api/ventas** Crear venta
- POST** **/api/ventas/{id}/confirmar-pago** Confirmar pago de venta
- GET** **/api/ventas/{id}** Obtener venta por ID
- GET** **/api/ventas/pendientes** Listar ventas pendientes de pago

### Notificaciones

Notificaciones personalizadas de promociones y cambios de pedido

- POST** **/api/notificaciones/promociones** Enviar promoción
- PATCH** **/api/notificaciones/{id}/leida** Marcar notificación como leída
- GET** **/api/notificaciones** Listar mis notificaciones

**Schemas**

DetalleVenta

## 6.2 HATEOAS (HAL)

Usamos assemblers Spring HATEOAS, por ejemplo:

- UsuarioModelAssembler (auth)
- ProductoModelAssembler, InventarioModelAssembler (catálogo)
- CarritoModelAssembler, VentaModelAssembler (ventas)

Las respuestas incluyen `_links` (self y relaciones de navegación) en formato HAL.

**Figura 13: Respuesta HAL con `_links` (ventas service)**

The screenshot shows a REST client interface for a 'MiniMarket Plus API'. The request is a POST to 'http://localhost:8080/api/carrito'. The response is a 200 OK status with a HAL (Hypertext Application Language) format. The response body is a JSON object with the following structure:

```

{
  "id": 1,
  "usuario": {
    "id": 4,
    "username": "cliente"
  },
  "items": [
    {
      "id": 1,
      "productoId": 1,
      "nombreProducto": "Leche entera 1L",
      "precio": 1200.0,
      "cantidad": 2
    }
  ],
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/carrito"
    },
    "carrito": {
      "href": "http://localhost:8080/api/carrito"
    },
    "agregarProducto": {
      "href": "http://localhost:8080/api/carrito"
    },
    "eliminarProducto": {
      "href": "http://localhost:8080/api/carrito/items/{productoId}",
      "templated": true
    },
    "vaciarCarrito": {
      "href": "http://localhost:8080/api/carrito"
    },
    "checkout": {
      "href": "http://localhost:8080/api/carrito/checkout"
    }
  }
}

```

The response also includes a 'Code snippet' section with a cURL command:

```

curl --location 'http://localhost:8080/api/carrito' \
--header 'Content-Type: application/json' \
--header 'Authorization: ....' \
--data '{
  "usuario": { "id": 4 },
  "producto": { "id": 1 },
  "cantidad": 2
}'

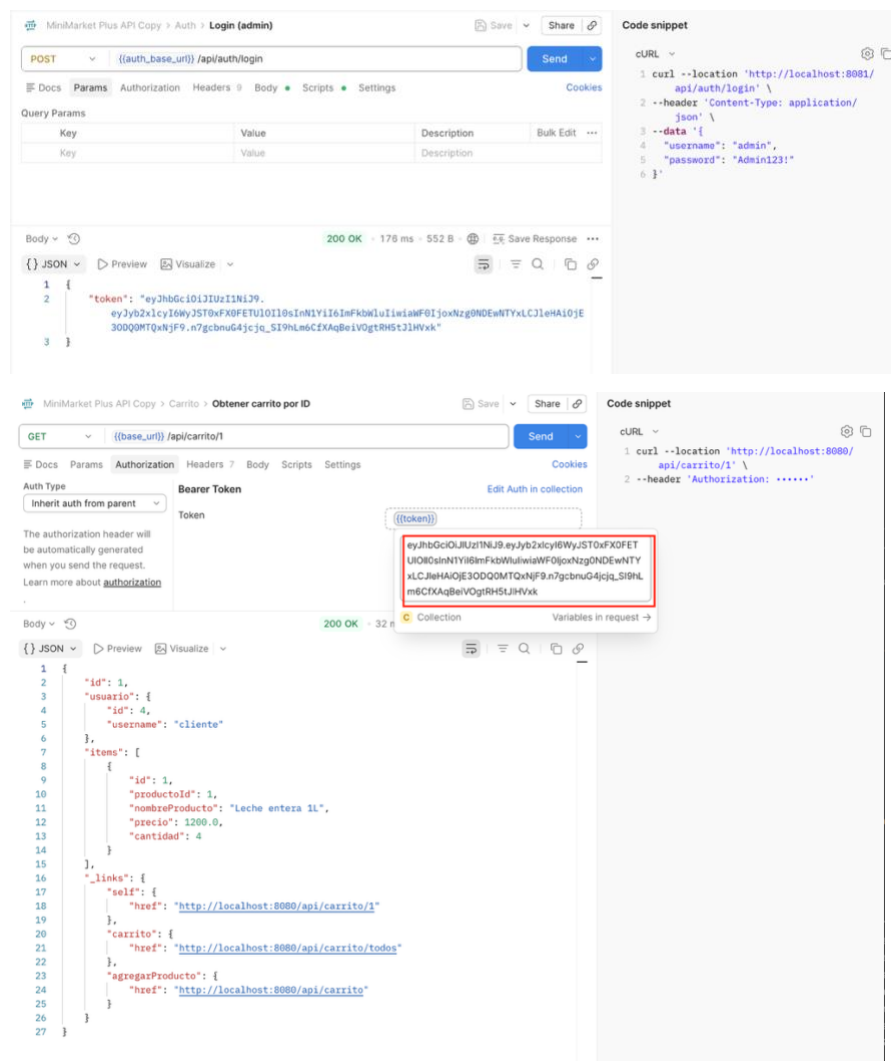
```

## 6.3 Colección Postman

Importamos docs/MiniMarket Plus API.postman\_collection.json con variables:

| Variable          | Default               | Uso                         |
|-------------------|-----------------------|-----------------------------|
| auth_base_url     | http://localhost:8081 | Login / registro / usuarios |
| catalogo_base_url | http://localhost:8082 | Productos / inventario      |
| base_url          | http://localhost:8080 | Carrito / ventas            |
| token             | (tras Login)          | Bearer de la colección      |

Figura 12: Postman — Login y uso del token



## 7. Reflexión y conclusiones

Pasar del monolito a tres servicios nos obligó a explicitar contratos (JWT compartido, RestClient de stock, sync interno de usuarios) y a no duplicar credenciales en ventas. Mantuvimos TDD y la suite de ~64 clases de test como red de seguridad al partir el código. OpenAPI por puerto y HATEOAS siguen siendo la cara pública de la API; la colección Postman cierra el circuito de demostración E2E.

Como limitaciones conscientes (YAGNI): Sin API Gateway por el momento ya que evitamos introducir complejidad innecesaria. Priorizamos un corte claro, seguro y demostrable para la EFT.

## 8. Referencias

| Recurso                              | Ubicación                                                                                                       |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Repositorio GitHub                   | <a href="https://github.com/gustavoduocuc/minimarket-plus">https://github.com/gustavoduocuc/minimarket-plus</a> |
| README (arranque, E2E curl)          | <a href="#">README.md</a>                                                                                       |
| Seguridad, roles, usuarios de prueba | <a href="#">docs/seguridad-acceso-y-datos.md</a>                                                                |
| Estados de venta                     | <a href="#">docs/estados-venta.md</a>                                                                           |
| Cobertura JaCoCo                     | <a href="#">docs/coverage.md</a>                                                                                |
| Colección Postman                    | <a href="#">docs/MiniMarket Plus API.postman_collection.json</a>                                                |
| Diagrama monolito                    | <a href="#">Diagrama1_architectura_antes_monolito.drawio</a>                                                    |
| Diagrama microservicios              | <a href="#">Diagrama2_architectura_actual_microservicios.drawio</a>                                             |
| SecurityConfig auth                  | <a href="#">auth-service/.../security/config/SecurityConfig.java</a>                                            |
| SecurityConfig ventas                | <a href="#">ventas-service/.../security/config/SecurityConfig.java</a>                                          |
| SecurityConfig inventario            | <a href="#">catalogo-inventario-service/.../security/.../SecurityConfig.java</a>                                |
| Ensure interno                       | <a href="#">ventas-service/.../controller/InternalUsuarioController.java</a>                                    |



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.